
Decision-Focused Learning via Tangent-Space Projection of Prediction Error

Junhyeong Lee¹ Sangjin Jin¹ Yongjae Lee^{1†}

Abstract

Decision-Focused Learning (DFL) trains predictors to improve downstream decision quality, but computing regret gradients typically requires differentiating through solvers or relying on surrogate losses, which can be computationally expensive or deviate from the true objective. We show that, under standard regularity with locally stable active constraints, the regret gradient admits a closed-form geometric characterization, equivalent to the prediction error projected onto the tangent space of active constraints, scaled by local curvature. This reveals that regret gradients can be obtained by filtering decision-irrelevant components from the MSE gradient, providing a simpler and more direct alternative to existing approaches. We propose **PEAR** (Projected Error As Regret-gradient), which computes regret gradients via a reduced linear system over active constraints, avoiding differentiation through solver iterations or additional optimization solves. Experiments on LP benchmarks and a real-world QP task show that PEAR achieves the best decision quality among all baselines while being the most computationally efficient, with gains that persist under constraint shifts. Code is available at [🔗](#).

1. Introduction

Machine learning is increasingly deployed in decision-making, where a predictive model estimates parameters of an optimization problem and a downstream solver computes the final decision. In these predict-then-optimize settings, reducing the prediction error alone does not necessarily yield good decisions. This has motivated Decision-Focused Learning (DFL) (Donti et al., 2017; Elmachoub & Grigas, 2022), which trains predictors to improve decision quality,

typically measured by regret, the suboptimality incurred by acting on predicted rather than true parameters.

Importantly, prediction accuracy and decision quality are not inherently at odds. In the linear-cost setting, minimizing mean squared error can be consistent with reducing expected regret (Elmachoub & Grigas, 2022). This raises a natural question about the structural relationship between prediction error and regret gradients, and how this relationship can inform training. However, this connection remains underexplored. Most prior work has instead focused on procedural questions, namely how to obtain regret gradients.

While such methods are valuable for enabling end-to-end learning, existing approaches have notable limitations. Among sensitivity-based differentiable optimization layers, general-purpose methods can trade off efficiency and robustness for broad applicability, and certain constructions require symmetrizing the KKT system, which can lead to ill-conditioned linear solves (Magoon et al., 2026; Bambade et al., 2024). A complementary line of work uses surrogate objectives or perturbed estimators to approximate regret gradients. These methods can provide informative learning signals, but they may deviate from the true regret objective, and many are tailored to linear-cost problems, which limits their generality beyond linear objectives (Mandi et al., 2024; Schutte et al., 2024).

We propose **PEAR** (Projected Error As Regret-gradient), a simple projection-based method for computing regret gradients. In contrast to prior work that mainly focuses on how to differentiate through an optimizer or design proxy objectives, we study the *structure* of regret gradients and their connection to prediction error. Under standard regularity and locally fixed active constraints, we show that the regret gradient admits an explicit geometric form. It is given by projecting the prediction error onto the tangent space of the active constraints under the local curvature metric. This result shows that the regret gradient can be obtained from the MSE gradient by filtering out decision-irrelevant components of the prediction error.

Guided by this characterization, PEAR computes regret gradients without differentiating through solver iterations and without relying on surrogate losses. Instead, it evaluates the required projection by solving a compact reduced linear system induced by the active constraints, e.g., via a Schur-

[†]Corresponding author. ¹Department of Industrial Engineering, Ulsan National Institute of Science and Technology, Ulsan, South Korea. Correspondence to: Yongjae Lee <yongjaelee@unist.ac.kr>.

complement computation. This avoids backward overhead and improves numerical stability compared to differentiable layers, while remaining faithful to the true regret objective rather than a proxy. PEAR applies to optimization problems with linear or quadratic objectives.

Our contributions are as follows:

1. **PEAR: a simple projection for regret gradients.** We establish an explicit geometric relationship between prediction error and regret gradients in Decision-Focused Learning. This interpretation reveals which error components influence decisions, and **PEAR** filters out decision-irrelevant directions to improve DFL performance.
2. **Efficient and stable computation.** PEAR evaluates the projection by solving a compact reduced linear system induced by the active constraints, reducing backward overhead and improving numerical stability compared to differentiable optimization layers. Compared to LP baselines, PEAR provides a deterministic regret gradient without requiring additional solver calls or perturbation-based estimation.
3. **Strong empirical performance.** On synthetic LP benchmarks and a real-world QP task, PEAR improves regret and training efficiency over other baselines in most settings, and remains strong under noise and constraint perturbations.

2. Related Work

Decision-focused learning. Decision-Focused Learning (DFL) trains predictors to improve downstream decision quality, typically measured by regret, in predict-then-optimize pipelines (Donti et al., 2017; Wilder et al., 2019; Elmachroub & Grigas, 2022). DFL has been studied in a range of applications including healthcare, portfolio optimization, and other optimization-driven decision systems (Verma et al., 2023; Costa & Iyengar, 2023; Lee et al., 2025; Hwang et al., 2025; Ellinas et al., 2024). Recent surveys summarize major methodological themes such as regularization-based approaches, convex surrogate losses, and perturbation-based gradient estimators (Mandi et al., 2024; Sadana et al., 2025). A recurring practical challenge is to obtain learning signals that are well aligned with regret while keeping training efficient, especially when training repeatedly interacts with a downstream optimization (Mandi et al., 2020; Mulamba et al., 2020; Berden et al., 2026).

Differentiable optimization layers. A common approach to end-to-end learning is to treat the downstream optimization as a differentiable layer. Early works relied on unrolling iterative optimization steps to compute gradients via the chain rule (Domke, 2012; Amos et al., 2017), while

OptNet (Amos & Kolter, 2017) introduced implicit differentiation through KKT conditions for quadratic programs. Subsequent work generalized this idea to broader classes of convex programs (Agrawal et al., 2019a;b; Gould et al., 2021; Blondel et al., 2022). While these methods provide principled gradients, in practice the backward pass depends heavily on the solver implementation. For instance, OptNet requires symmetrized linear systems that can become ill-conditioned (Bambade et al., 2024), while CVXPYLayers (Agrawal et al., 2019a) routes QPs through cone reformulations, restricting access to specialized solvers and degrading scalability on large sparse instances (Magoon et al., 2026). These issues make training expensive and limit solver choices (Blondel et al., 2022), motivating recent work on decoupling the forward and backward passes for solver-agnostic sensitivity analysis (Magoon et al., 2026).

Alternative approaches for regret minimization. Many real-world optimization problems are inherently discrete, leading to piecewise-constant solution maps that yield zero or undefined gradients (Mandi et al., 2022). To address uninformative gradients, DFL has largely focused on surrogate objectives and gradient approximations. For linear optimization, SPO+ (Elmachroub & Grigas, 2022) is a convex surrogate that upper bounds regret and can be optimized efficiently. A related strategy smooths the optimization problem itself, for example by adding quadratic regularization or using continuous relaxations to obtain meaningful gradients (Wilder et al., 2019; Mandi & Guns, 2020). Alternatively, perturbation-based methods keep the original solver but add random perturbations and use the resulting changes in the solution to build informative gradient estimates (Berthet et al., 2020; Pogančić et al., 2019; Niepert et al., 2021). These approaches can be practical and scalable, but they do not generally yield the true regret gradient and are often restricted to linear objectives, making generalization to broader problem classes difficult (Mandi et al., 2024; Schutte et al., 2024).

Sensitivity analysis and active set reduction. Our analysis builds on classical sensitivity theory for constrained optimization, which characterizes how solutions change under small perturbations (Fiacco, 1976; 1983; Robinson, 1980; Bonnans & Shapiro, 2013). Under standard regularity conditions, the active constraints remain locally stable and an inequality-constrained problem reduces to an equality-constrained system over the active set (Kyparisis, 1990; Dontchev & Rockafellar, 2009). This active set reduction yields efficient computations, where solution derivatives can be obtained from reduced linear systems defined by the active constraints (Bemporad et al., 2002). These principles enable differentiable optimization to exploit active constraint reduction across arbitrary solvers, computing gradients from reduced linear systems (Magoon et al., 2026).

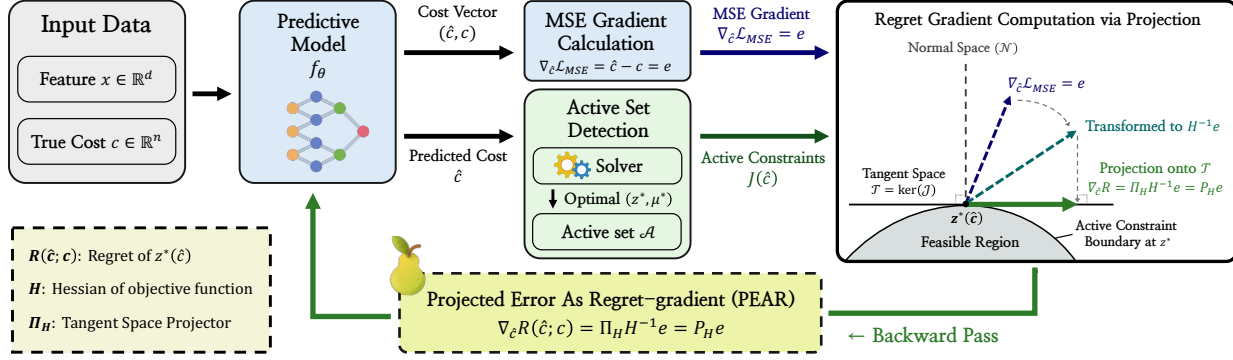


Figure 1. **The PEAR framework.** PEAR provides a direct pipeline for transforming prediction error into a regret gradient via a projection onto the local decision geometry. After identifying the active constraints, PEAR rescales the error by local curvature and projects it onto the feasible tangent space, filtering out decision-irrelevant components.

3. Regret Gradient via Tangent-Space Projection

This section establishes our main theoretical result. Under standard regularity conditions, the regret gradient admits a closed-form geometric characterization as the prediction error projected onto the tangent space of active constraints, scaled by local curvature. This structure reveals not only how to compute regret gradients efficiently by filtering decision-irrelevant components, but also why and what to correct for improving decisions.

3.1. Problem Setup

We consider a predict-then-optimize pipeline where contextual features $x \in \mathbb{R}^d$ inform predictions about unknown cost parameters. A predictive model $f_\theta : \mathbb{R}^d \rightarrow \mathbb{R}^n$ outputs $\hat{c} = f_\theta(x)$ as an estimate of the true cost vector $c \in \mathbb{R}^n$. Given $\hat{c}$, the downstream decision is obtained by solving the convex optimization problem

$$\begin{aligned} z^*(\hat{c}) \in \arg \min_{z \in \mathbb{R}^n} \quad & \phi(z) + \hat{c}^\top z \\ \text{subject to} \quad & Az = b, \\ & Gz \leq h, \end{aligned} \quad (1)$$

where $\phi : \mathbb{R}^n \rightarrow \mathbb{R}$ is a known convex base objective, $A \in \mathbb{R}^{p \times n}$ defines equality constraints, and $G \in \mathbb{R}^{m \times n}$ defines inequality constraints.

Decision-Focused Learning minimizes the regret of the decision computed from $\hat{c}$ under the true cost c :

$$\begin{aligned} \mathcal{R}(\hat{c}; c) = & [\phi(z^*(\hat{c})) + c^\top z^*(\hat{c})] \\ & - [\phi(z^*(c)) + c^\top z^*(c)]. \end{aligned} \quad (2)$$

Gradient-based training requires $\nabla_\theta \mathcal{R}$. By the chain rule,

$$\frac{d\mathcal{R}}{d\theta} = \frac{\partial \mathcal{R}}{\partial z^*} \cdot \frac{\partial z^*}{\partial \hat{c}} \cdot \frac{\partial \hat{c}}{\partial \theta}, \quad (3)$$

where the main challenge is the solution sensitivity $\partial z^* / \partial \hat{c}$, since it requires differentiating through an arg min operator (Wilder et al., 2019).

Since the ground-truth cost term $c^\top z^*(c)$ in (2) is independent of $\hat{c}$, the regret gradient with respect to $\hat{c}$ becomes

$$\nabla_{\hat{c}} \mathcal{R} = \left(\frac{\partial z^*}{\partial \hat{c}} \right)^\top [\nabla_{z^*} \phi(z^*) + c]. \quad (4)$$

3.2. Active Set and Local Regularity

The difficulty in differentiating $z^*(\hat{c})$ stems from inequality constraints. The mapping $\hat{c} \mapsto z^*(\hat{c})$ can be nonsmooth when the set of binding constraints changes. To recover local differentiability, we restrict attention to neighborhoods where the active set remains invariant. Within such a region, the inequality-constrained problem reduces to an equality-constrained one, in which inactive constraints are automatically satisfied and only the active constraints matter. Let $\mathcal{A}(\hat{c}) = \{i : (Gz^*(\hat{c}) - h)_i = 0\}$ denote the active set at $z^*(\hat{c})$ and define the stacked active constraint matrix

$$J(\hat{c}) = \begin{bmatrix} A \\ G_{\mathcal{A}(\hat{c})} \end{bmatrix} \in \mathbb{R}^{k \times n}, \quad k = p + |\mathcal{A}(\hat{c})|. \quad (5)$$

On any neighborhood where $\mathcal{A}(\hat{c})$ remains unchanged, the inequality-constrained problem behaves like an equality-constrained problem with constraint Jacobian J . To ensure the solution map remains smooth within this region, we require the following regularity conditions from classical sensitivity analysis (Fiacco, 1983).

Assumption 3.1 (Strict Convexity). The Hessian $H = \nabla^2 \phi(z^*)$ is positive definite.

Assumption 3.2 (Linear Independence Constraint Qualification). The active constraint Jacobian has full row rank, $\text{rank}(J) = k$.

Assumption 3.3 (Strict Complementary Slackness). For each active inequality $i \in \mathcal{A}$, the corresponding dual variable satisfies $\mu_i^* > 0$; for inactive constraints, $\mu_i^* = 0$.

Under these assumptions, the solution map is locally unique and continuous, and the active set remains invariant under sufficiently small perturbations of $\hat{c}$ (Robinson, 1980; Fiacco, 1983; Bonnans & Shapiro, 2013). A formal statement is provided in Appendix A.1.

3.3. Sensitivity via a Reduced KKT System

Fix $\hat{c}$ and consider perturbations $d\hat{c}$ that are small enough so that the active set does not change. At the optimal solution, the KKT conditions are given by

$$\begin{aligned} \nabla\phi(z^*) + \hat{c} + A^\top \lambda^* + G^\top \mu^* &= 0, \\ Az^* &= b, \quad Gz^* \leq h, \quad \mu^* \geq 0, \\ \text{diag}(\mu^*)(Gz^* - h) &= 0. \end{aligned} \quad (6)$$

Under Assumptions 3.1–3.3, inactive inequalities remain strictly feasible under sufficiently small perturbations that preserve the active set. Thus the problem locally reduces to an equality-constrained system whose KKT conditions become

$$\nabla\phi(z^*) + \hat{c} + J^\top y^* = 0, \quad Jz^* = \tilde{b}, \quad (7)$$

where J is the stacked active constraint Jacobian, $y^* \in \mathbb{R}^k$ collects dual variables for active constraints and $\tilde{b} = [b; h_A]$. We can view the reduced KKT system (7) as implicitly defining z^* and y^* as functions of $\hat{c}$. Define the implicit function

$$F(z, y, \hat{c}) = \begin{bmatrix} \nabla\phi(z) + \hat{c} + J^\top y \\ Jz - \tilde{b} \end{bmatrix} = 0. \quad (8)$$

By the implicit function theorem (Dontchev & Rockafellar, 2009), differentiating F with respect to $\hat{c}$ yields an explicit linear system for the perturbations dz and dy

$$\begin{bmatrix} H & J^\top \\ J & 0 \end{bmatrix} \begin{bmatrix} dz \\ dy \end{bmatrix} = \begin{bmatrix} -d\hat{c} \\ 0 \end{bmatrix}. \quad (9)$$

where $H = \nabla^2\phi(z^*)$ is the Hessian of the objective.

Since the active set is fixed, only the active constraints appear in the system, making this explicit formula much simpler than working with the full KKT conditions where many dual variables would be zero.

Under Assumptions 3.1–3.2, the KKT matrix in (9) is invertible (Boyd & Vandenberghe, 2004). Eliminating the dual variable dy via the Schur complement gives

$$dz = -P_H d\hat{c}, \quad \text{equivalently} \quad \frac{\partial z^*}{\partial \hat{c}} = -P_H, \quad (10)$$

where the sensitivity operator is

$$P_H = H^{-1} - H^{-1}J^\top(JH^{-1}J^\top)^{-1}JH^{-1}. \quad (11)$$

The derivation of (10)–(11) is standard in sensitivity analysis. For completeness, we include it in Appendix A.2.

The operator P_H enforces feasibility at first order. Indeed, substituting $dz = -P_H d\hat{c}$ into the linearized constraints yields $Jdz = 0$. Therefore, the first-order solution perturbations lie in the feasible tangent space $\mathcal{T} := \ker(J)$. This is made explicit by the projector identity below.

Proposition 3.4 (Projection Structure). *Under Assumptions 3.1–3.2, define*

$$\Pi_H = I - H^{-1}J^\top(JH^{-1}J^\top)^{-1}J. \quad (12)$$

Then Π_H is an H -orthogonal projector onto $\ker(J)$ and

$$P_H = \Pi_H H^{-1}, \quad JP_H = 0, \quad P_H J^\top = 0. \quad (13)$$

Proposition 3.4 shows that P_H can be read as curvature scaling followed by projection onto feasible directions. The proof is given in Appendix B.1.

3.4. Regret Gradient and the Projected Prediction Error

We now connect the sensitivity operator to the regret gradient. Let $e = \hat{c} - c$ denote the prediction error. Under Assumptions 3.1–3.3, the regret is differentiable with respect to $\hat{c}$ on any neighborhood where the active set is fixed. This allows the regret gradient in (4) to be simplified in closed form via the sensitivity operator P_H defined in (11).

Theorem 3.5 (Regret Gradient via Tangent-Space Projection). *Under Assumptions 3.1–3.3, the regret gradient admits the closed form*

$$\nabla_{\hat{c}}\mathcal{R}(\hat{c}; c) = P_H(\hat{c} - c) = P_H e,$$

where P_H is defined in (11).

Theorem 3.5 implies that regret descent replaces the MSE gradient e with a projected error $P_H e$ induced by the local active constraints and curvature. The proof combines the chain rule with the sensitivity relation (10) and the projection property in Proposition 3.4. A complete proof appears in Appendix B.2.

Since P_H projects onto $\mathcal{T} = \ker(J)$, it removes all normal components, leading to two key properties. First, the learning signal filters out normal directions. Second, it remains invariant to normal perturbations whenever the active set is fixed. These properties are formalized as follows.

Corollary 3.6 (Normal Filtering and Normal Invariance). *Let $\mathcal{N} = \text{range}(J^\top)$ denote the normal space to the tangent space $\mathcal{T}$. For any $\delta_N \in \mathcal{N}$,*

$$P_H \delta_N = 0. \quad (14)$$

If the active set remains unchanged, then for all $\alpha \in \mathbb{R}$,

$$\nabla_{\hat{c}}\mathcal{R}(\hat{c} + \alpha\delta_N; c) = \nabla_{\hat{c}}\mathcal{R}(\hat{c}; c). \quad (15)$$

A proof is given in Appendix B.3. Corollary 3.6 exposes the structural advantage of DFL. Prediction errors orthogonal to the feasible region do not affect the downstream decision, so DFL naturally suppresses such errors.

As shown in Figure 1, H^{-1} rescales the prediction error e according to the local curvature of the objective ϕ . In directions where the objective is flatter, smaller eigenvalues of H amplify the corresponding error components through $H^{-1}e$, while directions with steep curvature are correspondingly suppressed (Nocedal & Wright, 2006). This curvature-weighted error reveals which cost perturbations most affect the solution across different geometric directions.

The tangent projector then filters this rescaled error by projecting onto the feasible tangent space $\mathcal{T} = \ker(J)$, removing all components orthogonal to this space. The complete regret gradient is thus

$$\nabla_{\hat{c}}\mathcal{R}(\hat{c}; c) = \underbrace{(I - H^{-1}J^{\top}(JH^{-1}J^{\top})^{-1}J)}_{\text{tangent projector}} \underbrace{(H^{-1})}_{\text{curvature rescaler}} e,$$

which first rescales by local curvature, then projects onto the tangent space of feasible directions. This geometric decomposition is also characterized in Gould et al. (2021).

4. Computing the Projected-Error Gradient

This section describes how to compute the regret gradient $\nabla_{\hat{c}}\mathcal{R} = P_H e$ derived in Section 3 without explicitly forming the dense projection operator P_H . PEAR proceeds in three steps: (i) solve the forward problem to obtain primal and dual solutions, (ii) identify the active constraints and form the reduced Jacobian J , and (iii) compute $P_H e$ via a Schur-complement reduction over the active set.

4.1. Forward Pass: Active-Set Detection

We identify the active set from the primal–dual solution returned by the forward solver using a complementary slackness criterion. In our implementation, we solve (1) with OSQP (Stellato et al., 2020) after rewriting the constraints into $l \leq Gz \leq u$. Let $r^* = Gz^*$ denote the constraint residuals. We identify binding constraints via a complementary slackness test:

$$\begin{aligned} \mathcal{L} &= \{i \mid (r_i^* - l_i) < -y_i^*\}, \\ \mathcal{U} &= \{i \mid (u_i - r_i^*) < y_i^*\}, \end{aligned} \quad (16)$$

where $\mathcal{L}$ and $\mathcal{U}$ are the lower- and upper-active sets, and $\mathcal{A} = \mathcal{L} \cup \mathcal{U}$. Using $\mathcal{A}$, we construct the Jacobian J as in (5).

4.2. Backward Pass: Efficient Gradient Computation

Our goal in the backward pass is to compute the regret gradient $g = \nabla_{\hat{c}}\mathcal{R}(\hat{c}; c) = P_H e$ as derived in Theorem 3.5,

Algorithm 1 PEAR: Projected Error as Regret-gradient

Input: Predicted costs $\hat{c} = f_{\theta}(x)$, true costs c , problem data (A, b, G, h)
Input: Hessian $H = \nabla^2\phi(z^*)$ (or $H = \lambda I$ for LP), injection strength $\beta \in [0, 1]$
Output: Gradient signal $g = \nabla_{\hat{c}}\mathcal{R}(\hat{c}; c)$

- 1: **// Forward: Active-Set Detection**
- 2: Solve (1) with cost $\hat{c}$ to obtain primal-dual pair (z^*, y^*)
- 3: Identify active set $\mathcal{A}$ via complementary slackness (16)
- 4: Form active constraint Jacobian $J \leftarrow \begin{bmatrix} A \\ G_{\mathcal{A}} \end{bmatrix}$
- 5: **// Backward: Compute $P_H e$ via Reduced System**
- 6: Compute prediction error $e \leftarrow \hat{c} - c$
- 7: Compute $x \leftarrow H^{-1}e$ and $r \leftarrow Jx$
- 8: Solve reduced system $(JH^{-1}J^{\top})v = r$ for v
- 9: Compute projected gradient $g \leftarrow x - H^{-1}J^{\top}v$
- 10: **// Optional: Normal injection for LP**
- 11: **if** $\beta > 0$ **then**
- 12: Compute normal component $n \leftarrow H^{-1}J^{\top}v$
- 13: $g \leftarrow g + \beta \cdot \frac{\|g\|}{\|n\|} \cdot n$
- 14: **end if**
- 15: **return** g

where $e = \hat{c} - c$. A direct computation of $P_H e$ would form the dense $n \times n$ projection operator P_H explicitly, which becomes prohibitively expensive as dimension grows.

We instead compute $P_H e$ through a reduced linear system. The key insight is that under fixed active constraints, the primal perturbation dz can be eliminated, leaving a reduced system determined only by the active constraints. This yields the same projected vector without forming P_H .

Under the regularity conditions and using the reduced KKT sensitivity system in (9), we set the cost perturbation to the prediction error $d\hat{c} = -e$ and obtain

$$\begin{bmatrix} H & J^{\top} \\ J & 0 \end{bmatrix} \begin{bmatrix} dz \\ dy \end{bmatrix} = \begin{bmatrix} e \\ 0 \end{bmatrix}, \quad (17)$$

where $H = \nabla^2\phi(z^*)$ and J is the stacked Jacobian of the active constraints.

To eliminate dz , we use Schur complement reduction. From the first block of (17), we express

$$dz = H^{-1}(e - J^{\top}dy). \quad (18)$$

From the second block row of (17), we have $Jdz = 0$. Substituting (18) into this relation and defining $x = H^{-1}e$ and $r = Jx$, we obtain the reduced system

$$JH^{-1}J^{\top}dy = r. \quad (19)$$

This system has dimension $k \times k$ only and operates entirely within the reduced space of active multipliers, where $k =$

Table 1. **LP benchmarks.** Normalized regret (%), $\downarrow$ and training time (s), $\downarrow$, over 5 seeds. Best in **bold** and second best underlined.

Task	Method	Deg 2		Deg 4		Deg 6		Deg 8	
		Regret(%)	Time(s)	Regret(%)	Time(s)	Regret(%)	Time(s)	Regret(%)	Time(s)
Shortest Path	MSE	0.135 $\pm$ 0.037	7.5$\pm$0.8	1.967 $\pm$ 0.565	7.5$\pm$2.1	6.561 $\pm$ 1.514	8.9$\pm$1.5	15.428 $\pm$ 3.085	6.1$\pm$0.1
	SPO+	0.120 $\pm$ 0.046	48.7 $\pm$ 11.6	0.761$\pm$0.242	41.8 $\pm$ 10.8	<u>2.281$\pm$0.935</u>	44.1 $\pm$ 5.4	<u>4.403$\pm$1.491</u>	43.2 $\pm$ 7.5
	PFYL	0.177 $\pm$ 0.047	59.9 $\pm$ 3.2	0.946 $\pm$ 0.274	47.6 $\pm$ 12.0	2.435 $\pm$ 0.847	74.8 $\pm$ 22.9	5.236 $\pm$ 2.331	49.1 $\pm$ 12.1
	DBB	0.509 $\pm$ 0.109	116.4 $\pm$ 34.1	2.131 $\pm$ 0.597	139.3 $\pm$ 39.1	7.003 $\pm$ 1.936	98.7 $\pm$ 34.3	14.828 $\pm$ 3.086	85.2 $\pm$ 17.2
	LAVA	0.104$\pm$0.036	44.8 $\pm$ 8.1	0.851 $\pm$ 0.385	40.9 $\pm$ 3.9	2.483 $\pm$ 0.997	38.1 $\pm$ 2.4	6.189 $\pm$ 2.365	44.5 $\pm$ 6.0
	PEAR (Ours)	0.104$\pm$0.038	<u>28.4$\pm$5.4</u>	<u>0.774$\pm$0.268</u>	<u>23.5$\pm$0.4</u>	2.138$\pm$0.762	<u>35.1$\pm$4.9</u>	4.246$\pm$1.106	<u>38.4$\pm$2.5</u>
Knapsack	MSE	0.351 $\pm$ 0.020	422.8$\pm$44.5	0.922 $\pm$ 0.066	180.8$\pm$6.2	1.717 $\pm$ 0.226	111.7$\pm$11.8	2.285 $\pm$ 0.205	99.1$\pm$15.8
	SPO+	<u>0.291$\pm$0.015</u>	600.0 $\pm$ 0.0	<u>0.381$\pm$0.036</u>	600.0 $\pm$ 0.0	<u>0.608$\pm$0.066</u>	600.0 $\pm$ 0.0	<u>0.763$\pm$0.075</u>	600.0 $\pm$ 0.0
	PFYL	0.506 $\pm$ 0.025	600.0 $\pm$ 0.0	0.442 $\pm$ 0.042	600.0 $\pm$ 0.0	0.723 $\pm$ 0.049	600.0 $\pm$ 0.0	1.056 $\pm$ 0.147	600.0 $\pm$ 0.0
	DBB	1.852 $\pm$ 0.280	600.0 $\pm$ 0.0	0.910 $\pm$ 0.160	600.0 $\pm$ 0.0	1.040 $\pm$ 0.307	600.0 $\pm$ 0.0	1.081 $\pm$ 0.128	600.0 $\pm$ 0.0
	LAVA	0.438 $\pm$ 0.045	600.0 $\pm$ 0.0	0.478 $\pm$ 0.021	516.9 $\pm$ 81.1	0.820 $\pm$ 0.060	332.5 $\pm$ 111.0	1.351 $\pm$ 0.223	<u>182.8$\pm$42.5</u>
	PEAR (Ours)	0.287$\pm$0.007	<u>567.9$\pm$45.1</u>	0.315$\pm$0.020	<u>305.2$\pm$40.8</u>	0.388$\pm$0.032	<u>261.7$\pm$37.4</u>	0.437$\pm$0.043	271.7 $\pm$ 57.0

$p + |\mathcal{A}|$ consists of p equalities and $|\mathcal{A}|$ active inequalities. Solving for dy , the projected gradient is then recovered as

$$g = P_H e = x - H^{-1} J^\top dy, \quad (20)$$

with detailed derivation in Appendix B.4. In practice, we never form H^{-1} explicitly and instead apply H^{-1} through linear solves.

The computational advantage is substantial. Solving the $k \times k$ Schur complement system requires only matrix factorization and solving within the reduced space, whereas methods that differentiate through optimization solvers must handle full KKT systems. When the active set is sparse, this reduction yields significant efficiency gains without differentiating through solver iterations.

4.3. Linear Programs: Quadratic Smoothing and Normal Injection

In linear programming problems, the objective function lacks curvature and the solution map becomes piecewise constant. Over regions where the active set remains locally invariant, small cost perturbations do not alter the optimal solution. This creates plateaus where the regret gradient vanishes.

To obtain well-defined sensitivities while staying close to the original LP, we add a small quadratic regularization term $\phi(z) = \frac{\lambda}{2} \|z\|_2^2$ to our original problem (1), following Wilder et al. (2019). This yields $H = \lambda I \succ 0$, and the computation in Section 4.2 simplifies to

$$JJ^\top v = Je, \quad P_H e = \lambda^{-1}(e - J^\top v). \quad (21)$$

However, even after smoothing, the pure regret gradient $g = P_H e$ can remain weak and nearly constant over large regions in LPs (Mandi et al., 2025). To stabilize learning in

Table 2. **LP benchmarks under noise.** Normalized regret at degree 8 under varying noise levels.

Task	Method	Noise Level (ϵ)		
		0.1	0.3	0.5
Shortest Path	MSE	15.59 $\pm$ 2.66	17.08 $\pm$ 2.47	20.69 $\pm$ 2.95
	SPO+	4.74 $\pm$ 1.74	6.79 $\pm$ 2.56	10.49 $\pm$ 2.47
	PFYL	5.44 $\pm$ 2.20	7.21 $\pm$ 2.81	10.44 $\pm$ 2.83
	DBB	15.78 $\pm$ 3.85	17.95 $\pm$ 1.27	19.09 $\pm$ 1.89
	LAVA	6.60 $\pm$ 3.00	9.30 $\pm$ 3.78	12.46 $\pm$ 4.11
	PEAR (Ours)	4.50$\pm$1.86	5.88$\pm$2.04	10.38$\pm$2.54
Knapsack	MSE	2.33 $\pm$ 0.19	2.69 $\pm$ 0.19	3.24 $\pm$ 0.25
	SPO+	<u>0.81$\pm$0.07</u>	<u>1.12$\pm$0.06</u>	<u>1.77$\pm$0.10</u>
	PFYL	1.09 $\pm$ 0.15	1.40 $\pm$ 0.12	2.00 $\pm$ 0.13
	DBB	1.23 $\pm$ 0.17	1.52 $\pm$ 0.26	2.07 $\pm$ 0.12
	LAVA	1.43 $\pm$ 0.21	1.85 $\pm$ 0.27	2.93 $\pm$ 0.38
	PEAR (Ours)	0.49$\pm$0.05	0.86$\pm$0.05	1.58$\pm$0.08

these regimes and obtain a more informative update signal, we use normal injection. PEAR provides the same active set quantities used to compute g , and the corresponding normal component can be computed from (21) as $n = \lambda^{-1} J^\top v$. We then form a controlled combination

$$g_{\text{inj}} = g + \beta \cdot \frac{\|g\|}{\|n\|} n, \quad (22)$$

where $\beta \in [0, 1]$ controls the injection strength.

This update remains primarily driven by the regret signal, while selectively injecting normal space information when the tangent signal is weak. Scaling by $\|g\|$ preserves the relative magnitude of the injected term and prevents the normal correction from dominating the update. In flat regimes, this injection can enrich the gradient signal, helping the iterate reach neighborhoods where the active set changes and the regret signal becomes informative again.

Table 3. **Real-world QP.** Normalized regret (%), training time (s), and portfolio performance metrics over 5 seeds. Best in **bold** and second best best underlined.

Method	Regret(%)	Time(s)	Cum.Ret.(%)	Ann.Ret.(%)	Sharpe	MDD(%)
MSE	90.80 $\pm$ 2.05	33.0$\pm$9.5	89.22 $\pm$ 21.67	37.51 $\pm$ 6.18	0.92 $\pm$ 0.14	-40.00 $\pm$ 4.49
QPTH	87.85 $\pm$ 7.31	147.6 $\pm$ 42.4	<u>139.77$\pm$115.74</u>	<u>47.88$\pm$26.99</u>	<u>1.15$\pm$0.63</u>	<u>-34.94$\pm$14.51</u>
CvxpyLayers	<u>87.36$\pm$7.36</u>	321.9 $\pm$ 100.9	134.88 $\pm$ 79.13	47.39 $\pm$ 22.73	1.15 $\pm$ 0.52	-36.47 $\pm$ 6.76
PEAR (Ours)	85.38$\pm$6.91	<u>122.3$\pm$38.6</u>	184.19$\pm$86.24	58.84$\pm$19.49	1.44$\pm$0.47	-29.35$\pm$5.36

5. Experiments

This section evaluates PEAR on two synthetic LP benchmarks and a real-world QP task, measuring decision quality via normalized regret and training efficiency. Beyond standard benchmarks, we test robustness under cost noise and constraint shifts at test time. All results are reported over 5 random seeds. Details in Appendix C and E.

5.1. Synthetic LP Benchmarks

Setup. Our experimental setup follows Berden et al. (2026) for two standard LP tasks—Shortest Path and Knapsack—while testing on additional degree configurations and different problem instances. *Shortest Path* predicts edge costs on a 5×5 grid (40 edges) and solves for the minimum-cost path. *Knapsack* predicts item values for a 100-item 0–1 knapsack instance and selects an optimal subset under a capacity constraint. For Knapsack, methods requiring continuous sensitivity (ours and LAVA) train on the LP relaxation; we report regret on the original problem. The cost vectors for both tasks are generated from feature vectors $x \sim \mathcal{N}(0, I_p)$ via a polynomial mapping (Elmachtoub & Grigas, 2022; Tang & Khalil, 2022):

$$c_{ij} = \left[\frac{1}{3.5^{\deg}} \left(\frac{1}{\sqrt{p}} (\mathcal{B}x_i)_j + 3 \right)^{\deg} + 1 \right] \cdot \epsilon_{ij}, \quad (23)$$

where $\mathcal{B} \in \{0, 1\}^{d \times p}$ is a random binary matrix and p is the feature dimension. In the main experiments, we set $\epsilon_{ij} = 1$ (no noise). To evaluate robustness under model misspecification, we also test with multiplicative noise $\epsilon_{ij} \sim U(1 - \epsilon, 1 + \epsilon)$ for $\epsilon \in \{0.1, 0.3, 0.5\}$, which perturbs the cost-to-feature relationship and simulates scenarios where the true mapping is not perfectly captured by the polynomial model.

Baselines include MSE (two-stage) and representative DFL methods: SPO+ (Elmachtoub & Grigas, 2022), DBB (Pogančić et al., 2019), PFYL (Berthet et al., 2020), and LAVA (Berden et al., 2026). The first three are widely used baselines in the DFL literature, while LAVA is a recent solver-free approach that precomputes adjacent vertices of the LP polytope. All methods are implemented in PyEPO (Tang & Khalil, 2022) with recommended hyperparameter settings, using a linear predictor, Adam optimizer,

and early stopping when validation regret fails to improve by 1% for three consecutive evaluations. Training is capped at 600 seconds. Each task uses 1,000 training, 500 validation, and 500 test instances. We report normalized regret, defined as

$$\frac{\sum_{i=1}^{n_{\text{test}}} (c_i^\top z^*(\hat{c}_i) - c_i^\top z^*(c_i))}{\sum_{i=1}^{n_{\text{test}}} |c_i^\top z^*(c_i)|}. \quad (24)$$

Results. Table 1 reports normalized regret and training time across polynomial degrees. On Shortest Path, our method matches or outperforms all baselines at every degree, with larger gains as nonlinearity increases. At degree 4, SPO+ achieves marginally lower regret, but PEAR trains roughly twice as fast. On Knapsack, our method achieves the lowest regret across all configurations, with the margin over the next-best method widening as degree increases. In terms of training time, our approach is consistently among the fastest DFL methods. On Knapsack, several baselines reach the timeout without converging, whereas our method terminates earlier in most cases. At degree 8, LAVA is slightly faster despite being solver-free, but its regret is more than three times higher than ours. Table 2 evaluates robustness under multiplicative noise in the cost generation process, simulating scenarios where the mapping from features to costs is not perfectly specified. Even under high noise levels ($\epsilon = 0.5$), PEAR consistently achieves the lowest regret on both tasks. Overall, these results demonstrate that computing regret gradients via tangent-space projection yields strong decision quality and training efficiency, with robustness to noise in the cost generation process.

5.2. Real-World QP: Mean–Variance Optimization

Setup. We build on the experimental framework of Hwang et al. (2025) for mean–variance optimization on S&P 100 constituents over 2010–2024. The predictor estimates 21-day ahead expected returns from a 63-day lookback window using DLinear (Zeng et al., 2023). The downstream optimizer constructs a long-only portfolio with risk aversion $\lambda = 2.0$. We rebalance every 21 trading days and include a 0.1% turnover cost.

We compare against the two-stage MSE baseline and two differentiable QP layers, QPTH (Amos & Kolter, 2017) and CvxpyLayers (Agrawal et al., 2019a). We evaluate normal-

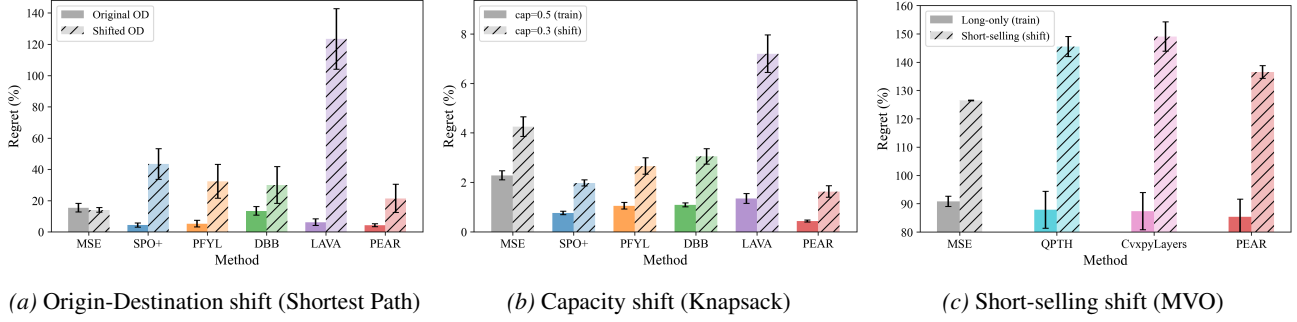


Figure 2. **Robustness under constraint shifts.** We train on an original train set and evaluate under shifted constraints. Our method maintains the lowest regret among DFL methods across all three tasks. Additional results are in Appendix F.

ized regret, cumulative and annualized returns, Sharpe ratio, and maximum drawdown.

Batched computation. In our implementation, the forward pass calls OSQP on each sample to obtain a primal-dual solution and identify the active constraints. We encode each active set as a binary mask indicating which inequality bounds are binding. The backward pass then proceeds entirely on GPU via batched linear algebra. Specifically, we (i) factorize $H = \lambda \Sigma$ via batched Cholesky decomposition, (ii) form the Schur complement using the binary masks, and (iii) solve for the dual variables and reconstruct $g = H^{-1}(e - J^T v)$. This decouples the forward solver from the backward computation, allowing efficient gradient computation without a differentiable optimization layer.

Results. Table 3 summarizes performance on real-world mean-variance optimization. PEAR attains the lowest normalized regret and translates this gain into stronger realized portfolios, achieving the highest cumulative and annualized returns, the lowest maximum drawdown, and the best Sharpe ratio. PEAR is also faster than differentiable QP layers. While all approaches are KKT-based, differentiable optimization layers can become unstable near active-set boundaries; PEAR instead computes the regret-aligned projection directly, yielding more stable gradients that likely explain the gap in out-of-sample performance. See Appendix F.4 for additional analysis on model behavior during downturns.

5.3. Robustness Under Constraint Shifts

Most DFL evaluations assume a fixed feasible region at train and test time. In practice, however, constraints often change after deployment due to resource limits, policy updates, or risk controls. To evaluate robustness, we train models at degree 8 (the highest-degree setting) and test under modified constraints while keeping the cost-generation process fixed. Figure 2 presents three complementary shifts. Details on the modified problem structures are provided in Appendix D.

In *Shortest Path* (a), we swap the origin and destination nodes symmetrically across the grid. Existing DFL methods degrade dramatically under this shift, suggesting they memorize optimal paths rather than learning transferable cost structure. Our method exhibits the smallest degradation among DFL baselines, while MSE appears most robust because it treats all edges equally during training.

In *Knapsack* (b), we tighten the capacity constraint from 0.5 to 0.3. Our method again achieves the lowest regret among DFL methods. Unlike Shortest Path, MSE degrades more substantially here, likely because knapsack decisions depend on value rankings rather than absolute cost predictions, an inductive bias that DFL methods capture but MSE does not.

In *MVO* (c), we allow short-selling at test time while training on long-only constraints. Unlike the previous two tasks, this shift expands the feasible region rather than restricting it. Among DFL methods, PEAR achieves the lowest regret, while MSE again shows the strongest robustness. However, the performance gap between methods is smaller than in the combinatorial settings, likely because all KKT-based methods compute similar gradients for the QP objective.

Overall, PEAR degrades the least among DFL baselines under all three constraint shifts and consistently maintains the lowest regret. See Appendix H for additional discussion.

6. Conclusion

We introduced PEAR, a framework for Decision-Focused Learning that obtains regret gradients by projecting prediction error onto the tangent space of active constraints. This geometric view reveals that the DFL signal is the MSE gradient filtered of decision-irrelevant components. By computing gradients via a reduced linear system rather than differentiating through solvers, PEAR achieves computational efficiency and numerical stability. Experiments on LP benchmarks and portfolio optimization demonstrate that PEAR achieves superior decision quality with reduced training time and improved robustness under constraint shifts.

Impact Statement

This work contributes to decision-focused learning by characterizing regret gradients geometrically and enabling their efficient computation without differentiating through optimization solvers. While such progress may have broad societal effects through downstream decision systems, we do not anticipate specific negative impacts beyond those common to predictive models used in optimization pipelines.

Acknowledgments

This work was supported by the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) (No. RS-2025-24803208) and the Institute of Information & Communications Technology Planning & Evaluation (IITP) grant funded by the Korea government (MSIT) (No. RS-2020-II201336, Artificial Intelligence Graduate School Program (UNIST)).

References

- Agrawal, A., Amos, B., Barratt, S., Boyd, S., Diamond, S., and Kolter, J. Z. Differentiable convex optimization layers. *Advances in neural information processing systems*, 32, 2019a.
- Agrawal, A., Barratt, S., Boyd, S., Busseti, E., and Moursi, W. M. Differentiating through a cone program. *arXiv preprint arXiv:1904.09043*, 2019b.
- Amos, B. and Kolter, J. Z. Optnet: Differentiable optimization as a layer in neural networks. In *International conference on machine learning*, pp. 136–145. PMLR, 2017.
- Amos, B., Xu, L., and Kolter, J. Z. Input convex neural networks. In *International conference on machine learning*, pp. 146–155. PMLR, 2017.
- Bambade, A., Schramm, F., Taylor, A., and Carpentier, J. Leveraging augmented-lagrangian techniques for differentiating over infeasible quadratic programs in machine learning. In *ICLR 2024-The Twelfth International Conference on Learning Representations*, 2024.
- Bemporad, A., Morari, M., Dua, V., and Pistikopoulos, E. N. The explicit linear quadratic regulator for constrained systems. *Automatica*, 38(1):3–20, 2002.
- Berden, S., Mahmutogullari, A., Tsouros, D., and Guns, T. Solver-free decision-focused learning for linear optimization problems. *Advances in Neural Information Processing Systems*, 38:158127–158145, 2026.
- Berthet, Q., Blondel, M., Teboul, O., Cuturi, M., Vert, J.-P., and Bach, F. Learning with differentiable perturbed optimizers. *Advances in neural information processing systems*, 33:9508–9519, 2020.
- Blondel, M., Berthet, Q., Cuturi, M., Frostig, R., Hoyer, S., Llinares-López, F., Pedregosa, F., and Vert, J.-P. Efficient and modular implicit differentiation. *Advances in neural information processing systems*, 35:5230–5242, 2022.
- Bonnans, J. F. and Shapiro, A. *Perturbation analysis of optimization problems*. Springer Science & Business Media, 2013.
- Boyd, S. and Vandenberghe, L. *Convex optimization*. Cambridge university press, 2004.
- Costa, G. and Iyengar, G. N. Distributionally robust end-to-end portfolio construction. *Quantitative Finance*, 23(10):1465–1482, 2023.
- Domke, J. Generic methods for optimization-based modeling. In *Artificial Intelligence and Statistics*, pp. 318–326. PMLR, 2012.
- Dontchev, A. L. and Rockafellar, R. T. *Implicit functions and solution mappings*, volume 543. Springer, 2009.
- Donti, P., Amos, B., and Kolter, J. Z. Task-based end-to-end model learning in stochastic optimization. *Advances in neural information processing systems*, 30, 2017.
- Ellinas, P., Kekatos, V., and Tsaousoglou, G. Decision-focused learning under decision dependent uncertainty for power systems with price-responsive demand. *Electric Power Systems Research*, 235:110665, 2024.
- Elmachtoub, A. N. and Grigas, P. Smart “predict, then optimize”. *Management Science*, 68(1):9–26, 2022.
- Fiacco, A. V. Sensitivity analysis for nonlinear programming using penalty methods. *Mathematical programming*, 10(1):287–311, 1976.
- Fiacco, A. V. *Introduction to sensitivity and stability analysis in non linear programming*. New York: Academic Press, 1983.
- Gould, S., Hartley, R., and Campbell, D. Deep declarative networks. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(8):3988–4004, 2021.
- Hwang, Y., Kong, Y., Zohren, S., and Lee, Y. Decision-informed neural networks with large language model integration for portfolio optimization. *arXiv preprint arXiv:2502.00828*, 2025.
- Kim, T., Kim, J., Tae, Y., Park, C., Choi, J.-H., and Choo, J. Reversible instance normalization for accurate time-series forecasting against distribution shift. In *International conference on learning representations*, 2021.

- Kyparisis, J. Sensitivity analysis for nonlinear programs and variational inequalities with nonunique multipliers. *Mathematics of operations research*, 15(2):286–298, 1990.
- Lee, J., Jeon, H., Bae, H., and Lee, Y. Return prediction for mean-variance portfolio selection: How decision-focused learning shapes forecasting models. In *Proceedings of the 6th ACM International Conference on AI in Finance*, pp. 114–122, 2025.
- Magoon, C., Yang, F., Aigerman, N., and Kovalsky, S. Differentiation through black-box quadratic programming solvers. *Advances in Neural Information Processing Systems*, 38:108486–108517, 2026.
- Mandi, J. and Guns, T. Interior point solving for lp-based prediction+ optimisation. *Advances in Neural Information Processing Systems*, 33:7272–7282, 2020.
- Mandi, J., Stuckey, P. J., Guns, T., et al. Smart predict-and-optimize for hard combinatorial optimization problems. In *Proceedings of the AAAI conference on artificial intelligence*, volume 34, pp. 1603–1610, 2020.
- Mandi, J., Bucarey, V., Tchomba, M. M. K., and Guns, T. Decision-focused learning: Through the lens of learning to rank. In *International conference on machine learning*, pp. 14935–14947. PMLR, 2022.
- Mandi, J., Kotary, J., Berden, S., Mulamba, M., Bucarey, V., Guns, T., and Fioretto, F. Decision-focused learning: Foundations, state of the art, benchmark and future opportunities. *Journal of Artificial Intelligence Research*, 80:1623–1701, 2024.
- Mandi, J., Mahmutogullari, A. İ., Berden, S., and Guns, T. Minimizing surrogate losses for decision-focused learning using differentiable optimization. *arXiv preprint arXiv:2508.11365*, 2025.
- Mulamba, M., Mandi, J., Diligenti, M., Lombardi, M., Bucarey, V., and Guns, T. Contrastive losses and solution caching for predict-and-optimize. *arXiv preprint arXiv:2011.05354*, 2020.
- Niepert, M., Minervini, P., and Franceschi, L. Implicit mle: backpropagating through discrete exponential family distributions. *Advances in Neural Information Processing Systems*, 34:14567–14579, 2021.
- Nocedal, J. and Wright, S. J. *Numerical Optimization*. Springer, New York, 2nd edition, 2006.
- Oberlin, C. and Wright, S. J. Active set identification in nonlinear programming. *SIAM Journal on Optimization*, 17(2):577–605, 2006.
- Pogančić, M. V., Paulus, A., Musil, V., Martius, G., and Rolinek, M. Differentiation of blackbox combinatorial solvers. In *International Conference on Learning Representations*, 2019.
- Robinson, S. M. Strongly regular generalized equations. *Mathematics of Operations Research*, 5(1):43–62, 1980.
- Sadana, U., Chenreddy, A., Delage, E., Forel, A., Frejinger, E., and Vidal, T. A survey of contextual optimization methods for decision-making under uncertainty. *European Journal of Operational Research*, 320(2):271–289, 2025.
- Schutte, N., Postek, K., and Yorke-Smith, N. Robust losses for decision-focused learning. In *33rd International Joint Conference on Artificial Intelligence*, pp. 4868–4875, 2024.
- Stellato, B., Banjac, G., Goulart, P., Bemporad, A., and Boyd, S. Osqp: An operator splitting solver for quadratic programs. *Mathematical Programming Computation*, 12(4):637–672, 2020.
- Tang, B. and Khalil, E. B. Pyepo: A pytorch-based end-to-end predict-then-optimize library for linear and integer programming. *arXiv preprint arXiv:2206.14234*, 2022.
- Verma, S., Mate, A., Wang, K., Madhiwalla, N., Hegde, A., Taneja, A., and Tambe, M. Restless multi-armed bandits for maternal and child health: Results from decision-focused learning. In *Proceedings of the 2023 International Conference on Autonomous Agents and Multiagent Systems*, pp. 1312–1320, 2023.
- Wilder, B., Dilkina, B., and Tambe, M. Melding the data-decisions pipeline: Decision-focused learning for combinatorial optimization. In *Proceedings of the AAAI conference on artificial intelligence*, volume 33, pp. 1658–1665, 2019.
- Zeng, A., Chen, M., Zhang, L., and Xu, Q. Are transformers effective for time series forecasting? In *Proceedings of the AAAI conference on artificial intelligence*, volume 37, pp. 11121–11128, 2023.

A. Additional Theory

A.1. Regularity and Local Differentiability

This appendix states standard conditions under which the solution mapping $\hat{c} \mapsto z^*(\hat{c})$ is locally single-valued and differentiable and the active set remains locally stable. Under Assumptions 3.1–3.3, classical sensitivity results imply that there exists a neighborhood of $\hat{c}$ on which the active set $\mathcal{A}(\hat{c})$ is constant and the KKT system admits a unique primal–dual solution; see, e.g., [Fiacco \(1983\)](#); [Bonnans & Shapiro \(2013\)](#). Consequently, on such a neighborhood the inequality-constrained problem locally behaves as the reduced equality-constrained problem in Section 3.3, and the implicit-function-based sensitivity analysis is valid.

A.2. Derivation of the Reduced KKT Sensitivity

For completeness, we derive Equations (10) and (11) from Equation (9) by block elimination. Let $S = JH^{-1}J^\top$. Since $H \succ 0$ and J has full row rank, S is invertible. From the first block row of Equation (9),

$$Hdz + J^\top dy = -d\hat{c} \quad \Rightarrow \quad dz = -H^{-1}d\hat{c} - H^{-1}J^\top dy.$$

Substituting into the second block row $Jdz = 0$ gives

$$-JH^{-1}d\hat{c} - JH^{-1}J^\top dy = 0 \quad \Rightarrow \quad dy = -S^{-1}JH^{-1}d\hat{c}.$$

Plugging back,

$$dz = -H^{-1}d\hat{c} + H^{-1}J^\top S^{-1}JH^{-1}d\hat{c} = -P_H d\hat{c},$$

with P_H as in Equation (11).

B. Proofs

B.1. Proof of Proposition 3.4

Proof. Let $S = JH^{-1}J^\top$, which is invertible under Assumption 3.1 and Assumption 3.2.

(i) We show $JP_H = 0$. By definition of P_H ,

$$\begin{aligned} JP_H &= JH^{-1} - JH^{-1}J^\top S^{-1}JH^{-1} \\ &= JH^{-1} - SS^{-1}JH^{-1} \\ &= JH^{-1} - JH^{-1} = 0. \end{aligned} \tag{25}$$

For $P_H J^\top = 0$, note that P_H is symmetric:

$$P_H^\top = H^{-1} - H^{-1}J^\top (JH^{-1}J^\top)^{-1}JH^{-1} = P_H, \tag{26}$$

since H^{-1} and S^{-1} are both symmetric. Thus $P_H J^\top = (JP_H)^\top = 0$.

(ii) For idempotency of Π_H :

$$\begin{aligned} \Pi_H^2 &= (I - H^{-1}J^\top S^{-1}J)^2 \\ &= I - 2H^{-1}J^\top S^{-1}J + H^{-1}J^\top S^{-1}JH^{-1}J^\top S^{-1}J \\ &= I - 2H^{-1}J^\top S^{-1}J + H^{-1}J^\top S^{-1}SS^{-1}J \\ &= I - 2H^{-1}J^\top S^{-1}J + H^{-1}J^\top S^{-1}J \\ &= I - H^{-1}J^\top S^{-1}J = \Pi_H. \end{aligned} \tag{27}$$

For H -self-adjointness, we show $\langle u, \Pi_H v \rangle_H = \langle \Pi_H u, v \rangle_H$ where $\langle a, b \rangle_H := a^\top H b$:

$$\begin{aligned} \langle u, \Pi_H v \rangle_H &= u^\top H \Pi_H v \\ &= u^\top H (I - H^{-1}J^\top S^{-1}J) v \\ &= u^\top (H - J^\top S^{-1}J) v. \end{aligned} \tag{28}$$

Since H and S^{-1} are symmetric, the matrix $H - J^\top S^{-1} J$ is symmetric, hence $\langle u, \Pi_H v \rangle_H = \langle \Pi_H u, v \rangle_H$.

(iii) Direct expansion:

$$\begin{aligned}\Pi_H H^{-1} &= (I - H^{-1} J^\top S^{-1} J) H^{-1} \\ &= H^{-1} - H^{-1} J^\top S^{-1} J H^{-1} = P_H.\end{aligned}\tag{29}$$

□

B.2. Proof of Theorem 3.5

Proof. Recall the regret function $\mathcal{R}(\hat{c}; c) = f(z^*(\hat{c}); c) - f(z^*(c); c)$, where $f(z; c) = \phi(z) + c^\top z$. Since $z^*(c)$ does not depend on $\hat{c}$, we have

$$\nabla_{\hat{c}} \mathcal{R}(\hat{c}; c) = \nabla_{\hat{c}} f(z^*(\hat{c}); c).$$

Step 1: Chain rule. On any neighborhood where the active set is fixed, the solution map is differentiable (Appendix A.1). As shown in Eq. (4), applying the chain rule yields

$$\begin{aligned}\nabla_{\hat{c}} \mathcal{R}(\hat{c}; c) &= \left(\frac{\partial z^*}{\partial \hat{c}} \right)^\top \nabla_z f(z; c) \Big|_{z=z^*(\hat{c})} \\ &= \left(\frac{\partial z^*}{\partial \hat{c}} \right)^\top (\nabla \phi(z^*) + c).\end{aligned}\tag{30}$$

Step 2: Substitute solution sensitivity. From Equation (10), $\frac{\partial z^*}{\partial \hat{c}} = -P_H$. Since P_H is symmetric (Appendix B.1), we obtain

$$\nabla_{\hat{c}} \mathcal{R}(\hat{c}; c) = -P_H (\nabla \phi(z^*) + c).$$

Step 3: Apply KKT stationarity. From the reduced KKT stationarity Equation (7),

$$\nabla \phi(z^*) + \hat{c} + J^\top y^* = 0 \quad \Rightarrow \quad \nabla \phi(z^*) + c = -(\hat{c} - c) - J^\top y^*.$$

Substituting gives

$$\nabla_{\hat{c}} \mathcal{R}(\hat{c}; c) = P_H(\hat{c} - c) + P_H J^\top y^*.$$

Step 4: Remove normal component. By Proposition 3.4, $P_H J^\top = 0$, so the second term vanishes and

$$\nabla_{\hat{c}} \mathcal{R}(\hat{c}; c) = P_H(\hat{c} - c) = P_H e.$$

□

B.3. Proof of Corollary 3.6

Proof. We prove the two statements separately.

Proof of Equation (14). Let $\delta_N \in \mathcal{N} = \text{range}(J^\top)$. Then there exists a vector v such that $\delta_N = J^\top v$. By the projection properties in Proposition 3.4, we have

$$P_H J^\top = 0,$$

and therefore

$$P_H \delta_N = P_H J^\top v = 0.$$

Proof of Equation (15). Assume that the active set remains unchanged when replacing $\hat{c}$ by $\hat{c} + \alpha \delta_N$ for any scalar α . Under this assumption, the same Jacobian J and projection operator P_H apply. By Theorem 3.5, the regret gradient is given by

$$\nabla_{\hat{c}} \mathcal{R}(\hat{c}; c) = P_H(\hat{c} - c).$$

Hence,

$$\nabla_{\hat{c}} \mathcal{R}(\hat{c} + \alpha \delta_N; c) = P_H((\hat{c} + \alpha \delta_N) - c) = P_H(\hat{c} - c) + \alpha P_H \delta_N.$$

Using Equation (14), the second term vanishes, which yields

$$\nabla_{\hat{c}} \mathcal{R}(\hat{c} + \alpha \delta_N; c) = P_H(\hat{c} - c).$$

This proves the invariance of the regret gradient under normal perturbations. $\square$

B.4. Proof of Section 4.2

Proof. We derive the reduced system by explicitly solving the linearized KKT system defined in Equation (17):

$$\begin{bmatrix} H & J^\top \\ J & 0 \end{bmatrix} \begin{bmatrix} dz \\ dy \end{bmatrix} = \begin{bmatrix} e \\ 0 \end{bmatrix}. \quad (31)$$

Writing out the block equations gives:

$$Hdz + J^\top dy = e, \quad (32)$$

$$Jdz = 0. \quad (33)$$

Since H is positive definite (Assumption 3.1), it is invertible. Solving (32) for the primal perturbation dz yields:

$$dz = H^{-1}(e - J^\top dy) = H^{-1}e - H^{-1}J^\top dy. \quad (34)$$

To eliminate dz , we substitute (34) into the constraint equation (33):

$$J(H^{-1}e - H^{-1}J^\top dy) = 0. \quad (35)$$

Rearranging terms to isolate the dual variable dy leads to the Schur complement system:

$$(JH^{-1}J^\top)dy = JH^{-1}e. \quad (36)$$

Letting $v = dy$ denote the dual perturbation, and defining the intermediate variables $x = H^{-1}e$ and $r = Jx$, this system becomes

$$(JH^{-1}J^\top)v = r, \quad (37)$$

which matches the reduced Schur complement system in Equation (19).

Finally, substituting v back into (34) yields $dz = H^{-1}e - H^{-1}J^\top v$. Since the solution of the linearized KKT system satisfies $dz = P_H e$, we identify the projected gradient as $g = P_H e = dz$, and hence

$$g = x - H^{-1}J^\top v, \quad (38)$$

which is Equation (20). Thus, solving the reduced Schur complement system suffices to compute $P_H e$ without explicitly forming P_H . $\square$

C. Optimization Problem Details

This appendix provides full specifications and data generation procedures for the benchmark problems and real-world portfolio optimization problem used in our experiments.

C.1. Shortest Path

We consider the standard shortest-path benchmark used in prior work (Elmachtoub & Grigas, 2022; Tang & Khalil, 2022). The problem is defined on a directed 5×5 grid graph $G = (V, E)$, where each node $(i, j) \in V$ is connected to its right and downward neighbors. The edge set is

$$E = \{((i, j), (i + 1, j)) \mid 0 \leq i < 4, 0 \leq j \leq 4\} \cup \{((i, j), (i, j + 1)) \mid 0 \leq i \leq 4, 0 \leq j < 4\},$$

yielding $|E| = 40$ edges and $|V| = 25$ nodes, with source $s = (0, 0)$ (top-left) and target $t = (4, 4)$ (bottom-right). Let c_{uv} be the cost of edge (u, v) and w_{uv} the decision variable indicating whether edge (u, v) lies on the path, and let $b \in \mathbb{R}^{|V|}$ be the demand vector with $b_s = -1$, $b_t = 1$, and $b_v = 0$ for every other node. The shortest path problem is the integer program whose linear relaxation is

$$\begin{aligned} \min_w \quad & \sum_{(u,v) \in E} c_{uv} w_{uv} \\ \text{subject to} \quad & \sum_{u: (u,v) \in E} w_{uv} - \sum_{u: (v,u) \in E} w_{vu} = b_v, \quad \forall v \in V \\ & 0 \leq w_{uv} \leq 1, \quad \forall (u,v) \in E \end{aligned} \quad (39)$$

The first sum runs over edges entering v and the second over edges leaving v , so the flow-balance constraints route one unit of flow from the source s to the target t .

C.2. Knapsack

We consider a binary knapsack problem with $n = 100$ items. Let $z_i = 1$ if item i is selected, and 0 otherwise. The problem is formulated as:

$$\begin{aligned} \max_z \quad & \sum_{i=1}^n c_i z_i \\ \text{subject to} \quad & \sum_{i=1}^n w_i z_i \leq C \\ & z_i \in \{0, 1\}, \quad \forall i \in \{1, \dots, n\} \end{aligned} \quad (40)$$

The objective maximizes the total value of selected items. The constraint ensures that the total weight does not exceed capacity C .

LP Relaxation. Since the original problem is an integer program, methods requiring continuous relaxation (PEAR and LAVA) solve the LP relaxation during training:

$$\begin{aligned} \max_z \quad & c^\top z \\ \text{subject to} \quad & w^\top z \leq C \\ & 0 \leq z_i \leq 1, \quad \forall i \in \{1, \dots, n\} \end{aligned} \quad (41)$$

All methods are evaluated on the original integer problem.

C.3. Mean-Variance Portfolio Optimization

We study portfolio optimization using historical daily returns from S&P 100 constituents over 2010–2024.

Problem Formulation. Let $w \in \mathbb{R}^n$ represent portfolio weights. The mean-variance optimization problem is:

$$\begin{aligned} \min_w \quad & \frac{\lambda}{2} w^\top \Sigma w - \mu^\top w \\ \text{subject to} \quad & \mathbf{1}^\top w = 1, \quad w \geq 0 \end{aligned} \quad (42)$$

where $\mu \in \mathbb{R}^n$ is the expected return vector, $\Sigma \in \mathbb{R}^{n \times n}$ is the covariance matrix, and $\lambda = 2.0$ is the risk aversion parameter. The constraints enforce a fully-invested long-only portfolio.

Data Construction. We use daily adjusted close prices to compute returns. The prediction model takes a 63-day lookback window of historical returns as input and forecasts 21-day ahead returns. Expected returns μ are computed as the mean of predicted daily returns. The covariance matrix Σ is estimated from the combined 84-day window (63-day history + 21-day prediction) with spectral shifting to ensure positive definiteness.

D. Optimization Problems for Constraint Shift Experiments

This appendix details the modified optimization problems used in the constraint shift experiments. These experiments evaluate how well DFL methods generalize when the feasible region changes between training and test time, while the cost structure remains the same.

D.1. Shortest Path: Direction Generalization

We additionally evaluate generalization across routing directions. Each configuration shares the 5×5 grid node set and the cost-generation procedure of Appendix C.1, but uses its own directed edge set and origin–destination pair. The optimization has the same form as the linear program in Appendix C.1, solved on the configuration-specific directed graph; the base problem is the forward configuration below.

Experimental Setup.

- **Forward (Training):** source $(0, 0)$, target $(4, 4)$, with edges directed rightward and downward.
- **Cross (Test):** source $(4, 0)$, target $(0, 4)$, with edges directed rightward and upward.

Only the directed edge set and the origin–destination pair differ between the two configurations; the grid nodes and the cost-generation procedure are identical. This tests whether learned edge-cost predictions transfer to a different routing direction on the same grid.

D.2. Knapsack: Capacity Generalization

We modify the knapsack problem from Appendix C.2 by varying the capacity constraint. The parameterized problem is:

$$\begin{aligned} \max_z \quad & \sum_{i=1}^n c_i z_i \\ \text{subject to} \quad & \sum_{i=1}^n w_i z_i \leq C(\rho) \\ & z_i \in \{0, 1\}, \quad \forall i \in \{1, \dots, n\} \end{aligned} \tag{43}$$

where the capacity is defined as a fraction of total weight:

$$C(\rho) = \rho \cdot \sum_{i=1}^n w_i. \tag{44}$$

Experimental Setup. Models are trained with $\rho_{\text{train}} = 0.5$ and evaluated on capacities $\rho_{\text{test}} \in \{0.3, 0.5, 0.7, 0.9\}$. Lower capacity ratios create tighter constraints (fewer items can fit), while higher ratios relax the constraint (more items can fit). This tests whether methods learn cost predictions that generalize across different constraint tightness levels.

D.3. Portfolio Optimization: Short Selling Generalization

We modify the mean-variance optimization problem from Appendix C.3 by relaxing the long-only constraint to allow short selling. The parameterized problem is:

$$\begin{aligned} \min_z \quad & \frac{\lambda}{2} z^\top \Sigma z - \mu^\top z \\ \text{subject to} \quad & \mathbf{1}^\top z = 1 \\ & z \geq \ell \end{aligned} \tag{45}$$

where $\ell \in \mathbb{R}$ is the lower bound on individual asset weights.

Experimental Setup. Models are trained with $\ell_{\text{train}} = 0$ (long-only portfolio) and evaluated on $\ell_{\text{test}} \in \{-0.1, -0.3, -0.5, -1.0\}$. Negative lower bounds allow short positions up to the specified limit. For example, $\ell = -0.1$ permits shorting up to 10% of portfolio value in any single asset. This tests whether return predictions learned under long-only constraints remain useful when the optimization gains additional flexibility through short selling.

D.4. Summary of Constraint Shifts

Table 4 summarizes the constraint modifications across all three problem domains.

Table 4. Summary of constraint shift experiments. All experiments use the same cost/return prediction model trained under the training constraint, evaluated across multiple test constraints.

Problem	Modified Constraint	Training	Test
Knapsack	Capacity ratio ρ	0.5	$\{0.3, 0.5, 0.7, 0.9\}$
Shortest Path	Origin-destination pair (v_s, v_t)	$(0, 0) \rightarrow (4, 4)$	$(0, 4) \rightarrow (4, 0)$
Portfolio	Weight lower bound ℓ	0	$\{-0.1, -0.3, -0.5, -1.0\}$

E. Experimental Details

Our code is available at <https://github.com/FinJun/PEAR>. All experiments are conducted on a single NVIDIA RTX A5000 GPU with an Intel Xeon Silver 4210R CPU, and repeated over 5 random seeds $\{0, 1, 2, 3, 4\}$ with mean and standard deviation reported.

E.1. LP benchmarks Experiments (Shortest Path, Knapsack)

Following Elmachtoub & Grigas (2022), we generate 1,000 training, 500 validation, and 500 test samples, varying the polynomial degree $d \in \{2, 4, 6, 8\}$ to control problem difficulty. We use a single-layer linear network mapping 5-dimensional features to edge costs (40 dimensions for Shortest Path) or item values (100 dimensions for Knapsack). The parameter β for normal injection in PEAR was fixed at 0.1 throughout all experiments.

E.2. Real-world QP Experiments (Portfolio Optimization)

We use a chronological 70:10:20 split for train, validation, and test sets. The prediction model is DLinear (Zeng et al., 2023) with RevIN (Kim et al., 2021) normalization, using a lookback window of $L = 63$ days and prediction horizon of $H = 21$ days. All methods are trained with the Adam optimizer using learning rate 10^{-3} and batch size 64, with learning rate reduction (factor 0.5, patience 3) and early stopping (patience 10) based on validation regret.

For evaluation, portfolios are rebalanced every 21 trading days with a transaction cost of 0.1% applied to turnover. We report cumulative return $\prod_{t=1}^T (1 + r_t) - 1$, annualized return and volatility (scaled by $\sqrt{252}$), Sharpe ratio assuming zero risk-free rate, maximum drawdown, and average turnover at each rebalancing.

F. Additional Results

We evaluate generalization under constraint shifts, where test-time constraints differ from training. This setting is practically important since real-world deployment often involves changing requirements.

F.1. Capacity Shifts (Knapsack)

We train models with capacity ratio 0.5 and evaluate on ratios $\{0.3, 0.5, 0.7, 0.9\}$. As shown in Table 5, PEAR consistently achieves the best performance across all polynomial degrees and capacity ratios. Surrogate-based methods (SPO+, PFYL) degrade significantly at higher capacities (0.9), while PEAR maintains robust performance. DBB exhibits particularly poor generalization at low polynomial degrees.

F.2. Direction Shifts (Shortest Path)

We train models on one source-target pair and evaluate on a different direction. Table 6 reveals an interesting pattern: MSE achieves the best performance across all degrees, with PEAR as the second best. This suggests that task-specific DFL objectives can overfit to training constraint structures, whereas prediction-focused training (MSE) and geometry-aware methods (PEAR) generalize better to unseen constraint configurations.

Table 5. **Capacity generalization (Knapsack)**. Normalized regret ($\%$, $\downarrow$) under varying capacity ratios. Best in **bold**, second best underlined. *Training capacity.

Method	Degree 2				Degree 4			
	0.3	0.5*	0.7	0.9	0.3	0.5*	0.7	0.9
MSE	0.449 $\pm$ 0.011	0.350 $\pm$ 0.017	0.255 $\pm$ 0.014	0.141 $\pm$ 0.014	1.269 $\pm$ 0.082	0.922 $\pm$ 0.059	1.037 $\pm$ 0.123	1.557 $\pm$ 0.219
SPO+	0.373$\pm$0.006	<u>0.291$\pm$0.013</u>	0.260 $\pm$ 0.012	0.444 $\pm$ 0.056	0.542$\pm$0.034	<u>0.381$\pm$0.032</u>	0.834 $\pm$ 0.087	2.520 $\pm$ 0.216
PFYL	0.610 $\pm$ 0.024	0.506 $\pm$ 0.022	0.937 $\pm$ 0.103	5.109 $\pm$ 0.428	0.628 $\pm$ 0.060	0.441 $\pm$ 0.038	0.922 $\pm$ 0.145	3.792 $\pm$ 0.620
DBB	2.514 $\pm$ 0.264	1.852 $\pm$ 0.250	2.093 $\pm$ 0.217	6.155 $\pm$ 0.629	1.003 $\pm$ 0.118	0.910 $\pm$ 0.143	1.907 $\pm$ 0.213	5.542 $\pm$ 0.342
LAVA	0.636 $\pm$ 0.051	0.439 $\pm$ 0.039	0.537 $\pm$ 0.021	2.191 $\pm$ 0.186	1.346 $\pm$ 0.074	0.477 $\pm$ 0.018	<u>0.644$\pm$0.053</u>	3.337 $\pm$ 0.770
PEAR	<u>0.420$\pm$0.041</u>	0.287$\pm$0.006	0.222$\pm$0.010	0.140$\pm$0.029	<u>0.589$\pm$0.057</u>	0.315$\pm$0.017	0.355$\pm$0.029	0.590$\pm$0.254
Method	Degree 6				Degree 8			
	0.3	0.5*	0.7	0.9	0.3	0.5*	0.7	0.9
MSE	2.629 $\pm$ 0.303	1.717 $\pm$ 0.202	1.946 $\pm$ 0.164	<u>2.909$\pm$0.171</u>	4.252 $\pm$ 0.394	2.285 $\pm$ 0.183	2.036 $\pm$ 0.182	<u>2.732$\pm$0.204</u>
SPO+	<u>1.187$\pm$0.066</u>	<u>0.608$\pm$0.059</u>	1.225 $\pm$ 0.109	3.416 $\pm$ 0.233	<u>1.975$\pm$0.123</u>	<u>0.763$\pm$0.067</u>	<u>1.194$\pm$0.117</u>	3.067 $\pm$ 0.243
PFYL	<u>1.528$\pm$0.082</u>	<u>0.723$\pm$0.044</u>	1.165 $\pm$ 0.254	3.783 $\pm$ 1.056	2.658 $\pm$ 0.331	1.056 $\pm$ 0.132	1.255 $\pm$ 0.113	3.308 $\pm$ 0.351
DBB	1.437 $\pm$ 0.298	1.040 $\pm$ 0.275	2.026 $\pm$ 0.330	4.864 $\pm$ 0.472	2.156 $\pm$ 0.220	1.081 $\pm$ 0.114	1.849 $\pm$ 0.117	4.256 $\pm$ 0.218
LAVA	3.742 $\pm$ 0.259	0.820 $\pm$ 0.053	<u>0.996$\pm$0.228</u>	4.721 $\pm$ 1.124	7.196 $\pm$ 0.758	1.351 $\pm$ 0.200	1.541 $\pm$ 0.498	5.397 $\pm$ 1.086
PEAR	0.986$\pm$0.088	0.388$\pm$0.029	0.478$\pm$0.036	1.043$\pm$0.224	1.628$\pm$0.233	0.437$\pm$0.038	0.554$\pm$0.058	1.232$\pm$0.197

Table 6. **Direction generalization (Shortest Path)**. Normalized regret ($\%$, $\downarrow$) under constraint shift. Best in **bold**, second best underlined.

Method	Degree 2	Degree 4	Degree 6	Degree 8
MSE	0.14$\pm$0.03	1.95$\pm$0.44	6.53$\pm$1.30	14.00$\pm$1.47
SPO+	9.38 $\pm$ 4.40	22.14 $\pm$ 8.43	29.03 $\pm$ 6.71	43.40 $\pm$ 9.83
PFYL	9.38 $\pm$ 2.37	21.05 $\pm$ 6.26	23.35 $\pm$ 6.15	32.33 $\pm$ 10.89
DBB	3.59 $\pm$ 0.72	8.95 $\pm$ 6.08	22.55 $\pm$ 8.69	31.06 $\pm$ 5.78
LAVA	16.84 $\pm$ 3.12	45.01 $\pm$ 8.40	78.73 $\pm$ 13.33	123.40 $\pm$ 19.35
PEAR	<u>0.32$\pm$0.37</u>	<u>3.63$\pm$2.08</u>	<u>7.07$\pm$1.90</u>	<u>21.42$\pm$9.02</u>

E.3. Lower Bound Shifts (Portfolio Optimization)

We train models with long-only constraints ($lb = 0$) and evaluate under relaxed lower bounds allowing short-selling. Table 7 shows that MSE achieves the best generalization, with PEAR as the closest DFL method. Differentiable optimization methods (QPTH, CvxpyLayers) show degraded performance as constraints deviate further from training conditions.

Table 7. **Lower bound generalization (MVO)**. Normalized regret ($\%$, $\downarrow$) under varying lower bound constraints. Best in **bold**, second best underlined.

Method	lb=-0.1	lb=-0.3	lb=-0.5	lb=-1.0
MSE	119.9$\pm$0.2	126.4$\pm$0.1	130.6$\pm$0.1	138.3$\pm$0.1
QPTH	129.3 $\pm$ 2.4	145.5 $\pm$ 4.0	152.4 $\pm$ 5.7	162.4 $\pm$ 6.6
CvxpyLayers	129.8 $\pm$ 2.5	149.0 $\pm$ 5.8	156.9 $\pm$ 6.8	167.6 $\pm$ 7.1
PEAR	<u>124.1$\pm$3.4</u>	<u>136.5$\pm$2.5</u>	<u>142.6$\pm$1.7</u>	<u>153.4$\pm$0.9</u>

E.4. Portfolio Performance Analysis

Beyond regret minimization (Table 3), we evaluate whether PEAR’s gradient estimation translates to superior real-world portfolio performance. In mean-variance optimization, balancing returns against risk is crucial—high returns alone are insufficient if the portfolio exhibits excessive volatility during market turbulence. Figure 3 shows cumulative portfolio value over the out-of-sample period, where PEAR achieves the highest terminal value, demonstrating that improved training regret translates to superior realized returns. More importantly, PEAR’s advantage extends beyond raw returns. Figure 4 examines portfolio behavior during the two most severe drawdown periods (days 80–180 and 560–640). In both cases, PEAR exhibits the most stable trajectory, demonstrating superior robustness to market downturns compared to differentiable optimization methods. This suggests that PEAR learns predictions that better capture the risk-return trade-off encoded in the

mean-variance objective.

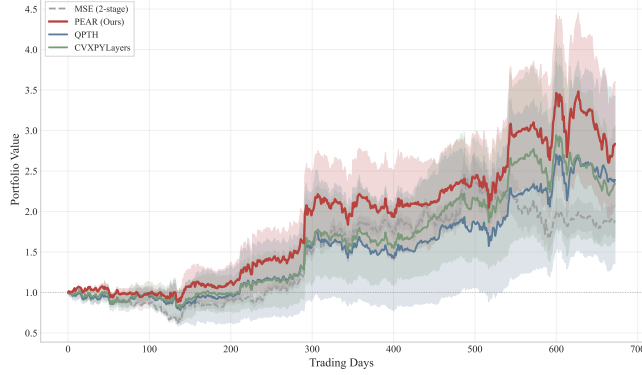


Figure 3. **Portfolio cumulative returns.** Portfolio value over the test period. PEAR achieves the highest value among all methods.

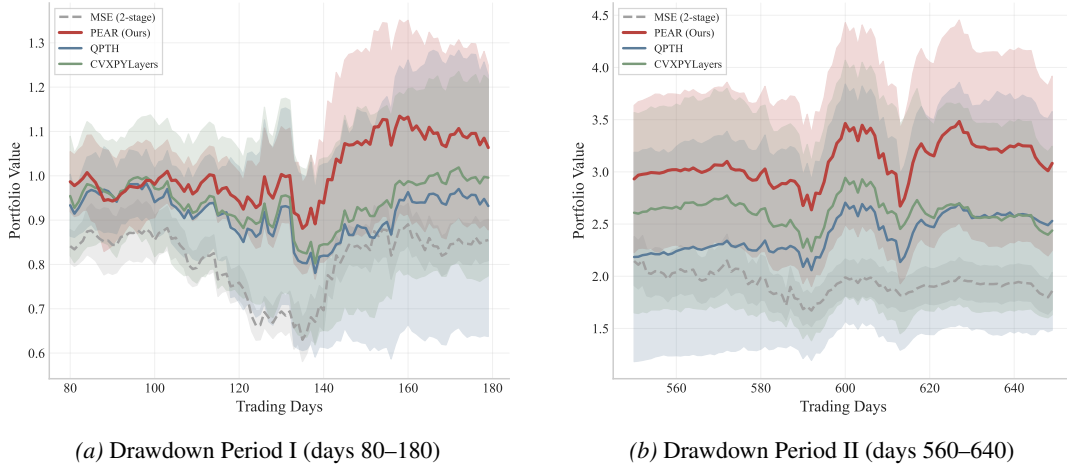


Figure 4. **Drawdown analysis.** Portfolio value during the two most severe market downturns. PEAR demonstrates the most stable behavior, indicating robust risk management.

G. Additional Experiments

This appendix presents additional empirical results that complement those in the main paper. Appendix G.1 analyzes sensitivity to PEAR’s two LP-specific hyperparameters, and Appendix G.2 evaluates PEAR on a real-world knapsack.

G.1. Hyperparameter Ablations on LP Benchmarks

For LP problems, PEAR introduces two hyperparameters, the quadratic smoothing strength λ and the normal injection coefficient β . We isolate the sensitivity of PEAR to each on the two LP benchmarks. All other settings follow Appendix E.1, and we report normalized regret over 5 seeds. The default used in the main experiments is $(\lambda, \beta) = (0.1, 0.1)$.

Normal Injection. We sweep $\beta \in \{0, 0.05, 0.1, 0.2, 0.5\}$ with $\lambda = 0.1$ fixed. Table 8 reports the normalized regret. On Shortest Path the choice of β has only a mild effect. $\beta = 0$ is already competitive and attains the best regret at the hardest degree (Deg 8), while small positive β is marginally better at the intermediate degrees. On Knapsack the effect is more pronounced—a small amount of normal information consistently helps, with $\beta = 0.05$ best at the higher degrees (Deg 6 and 8) and $\beta = 0.1$ best at Deg 4, whereas the pure gradient is the weakest of the small- β settings. Large β (0.5) degrades performance on Knapsack, worsening monotonically with the polynomial degree. PEAR is otherwise stable for $\beta \in [0.05, 0.2]$, and we use $\beta = 0.1$ as the default.

Table 8. **Normal-injection ablation.** Normalized regret (% , $\downarrow$) under varying β , with $\lambda = 0.1$ fixed. Best β per degree in **bold**, second best underlined.

Task	β	Deg 2	Deg 4	Deg 6	Deg 8
Shortest Path	0	0.111 $\pm$ 0.030	0.777 $\pm$ 0.216	2.314 $\pm$ 0.997	4.215$\pm$1.294
	0.05	0.119 $\pm$ 0.014	0.751$\pm$0.226	2.257 $\pm$ 1.019	5.763 $\pm$ 3.971
	0.1	0.104 $\pm$ 0.034	0.774 $\pm$ 0.239	2.138$\pm$0.681	4.246 $\pm$ 0.989
	0.2	0.112 $\pm$ 0.035	0.801 $\pm$ 0.249	<u>2.162$\pm$0.726</u>	4.574 $\pm$ 1.915
	0.5	0.096$\pm$0.033	0.811 $\pm$ 0.239	2.277 $\pm$ 0.963	4.329 $\pm$ 1.633
Knapsack	0	0.299 $\pm$ 0.014	0.346 $\pm$ 0.019	0.429 $\pm$ 0.039	0.477 $\pm$ 0.039
	0.05	0.288 $\pm$ 0.019	<u>0.321$\pm$0.018</u>	0.378$\pm$0.025	0.406$\pm$0.042
	0.1	0.287 $\pm$ 0.006	0.315$\pm$0.017	<u>0.388$\pm$0.029</u>	<u>0.437$\pm$0.038</u>
	0.2	0.283$\pm$0.010	0.332 $\pm$ 0.019	<u>0.432$\pm$0.026</u>	<u>0.495$\pm$0.042</u>
	0.5	<u>0.286$\pm$0.014</u>	0.366 $\pm$ 0.025	0.533 $\pm$ 0.040	0.653 $\pm$ 0.057

QP Smoothing. We sweep $\lambda \in \{0.01, 0.05, 0.1, 0.5, 1.0\}$ with $\beta = 0.1$ fixed. Small λ keeps the forward solution close to the original LP but yields nearly singular projections; large λ stabilizes the projection but biases the forward solution away from the LP optimum. Table 9 reports the normalized regret. On Shortest Path, $\lambda = 0.1$ is optimal at every degree, while $\lambda = 0.5$ is preferred on Knapsack. Across the grid, $\lambda \in [0.1, 0.5]$ stays close to the best regret, and we use $\lambda = 0.1$ as the default.

Table 9. **QP-smoothing ablation.** Normalized regret (% , $\downarrow$) under varying λ , with $\beta = 0.1$ fixed. Best λ per degree in **bold**, second best underlined.

Task	λ	Deg 2	Deg 4	Deg 6	Deg 8
Shortest Path	0.01	0.471 $\pm$ 0.281	1.834 $\pm$ 0.803	4.554 $\pm$ 1.838	9.410 $\pm$ 3.792
	0.05	0.152 $\pm$ 0.049	0.898 $\pm$ 0.301	2.421 $\pm$ 0.859	5.906 $\pm$ 2.529
	0.1	0.104$\pm$0.034	0.774$\pm$0.239	2.138$\pm$0.681	4.246$\pm$0.989
	0.5	<u>0.114$\pm$0.044</u>	1.221 $\pm$ 0.325	3.078 $\pm$ 0.964	<u>4.952$\pm$1.506</u>
	1.0	0.128 $\pm$ 0.029	1.384 $\pm$ 0.329	3.992 $\pm$ 1.224	6.878 $\pm$ 2.462
Knapsack	0.01	0.361 $\pm$ 0.007	0.386 $\pm$ 0.024	0.521 $\pm$ 0.021	0.709 $\pm$ 0.110
	0.05	0.293 $\pm$ 0.011	0.329 $\pm$ 0.021	0.419 $\pm$ 0.025	0.473 $\pm$ 0.045
	0.1	0.287 $\pm$ 0.006	<u>0.315$\pm$0.017</u>	<u>0.388$\pm$0.029</u>	0.437 $\pm$ 0.038
	0.5	0.268$\pm$0.014	0.311$\pm$0.019	0.370$\pm$0.027	0.391$\pm$0.029
	1.0	<u>0.269$\pm$0.010</u>	0.322 $\pm$ 0.021	0.409 $\pm$ 0.029	<u>0.429$\pm$0.033</u>

G.2. Real-World Knapsack: Building Investment

Real-world knapsack benchmarks are uncommon in the DFL literature, where the standard practice is to use synthetic feature–cost mappings as in our LP benchmarks above. For completeness, we additionally report results on the building investment dataset of Mandi & Guns (2020), one of the few real-world knapsack instances used in prior DFL work, with 372 properties labeled by construction cost and sales price.

Problem Formulation. Each test instance solves a binary knapsack with predicted sales prices as item values and known construction costs as weights.

$$\begin{aligned}
 & \max_z \quad \sum_{i=1}^n \hat{c}_i z_i \\
 & \text{subject to} \quad \sum_{i=1}^n w_i z_i \leq B \\
 & \quad z_i \in \{0, 1\}, \quad \forall i \in \{1, \dots, n\}
 \end{aligned} \tag{46}$$

Here $\hat{c}_i$ is the predicted sales price of property i , w_i is its construction cost, and B is the budget. We follow the setup of Mandi & Guns (2020) with two changes. We use an inequality budget constraint instead of equality, which avoids

infeasibility across batch sizes, and we use a batch size of 10 instead of 31. The budget is set to $B = 0.4 \sum_i w_i$, all other settings follow Appendix E.1, and we report normalized regret over 5 seeds.

Table 10. **Real-world Knapsack (building investment)**. Normalized regret (% , $\downarrow$) over 5 seeds. Best in **bold**, second best underlined.

Method	MSE	SPO+	PFYL	DBB	LAVA	PEAR
Regret (%)	1.22 $\pm$ 0.89	0.72 $\pm$ 0.57	<u>0.70$\pm$0.80</u>	12.20 $\pm$ 10.05	2.12 $\pm$ 0.79	0.51$\pm$0.57

Table 10 reports the normalized regret. PEAR achieves the lowest regret, with PFYL second. The two-stage MSE baseline trails the leading DFL methods, and DBB shows the largest regret with substantial variance.

H. Additional Discussion

Stability of Active-Set Identification. Our theoretical results rely on correctly identifying the active constraint set at the optimal solution, which in practice is inferred from a numerical solver and can be sensitive near constraint boundaries; thus constraints that are close to active can be misclassified (Oberlin & Wright, 2006). When this happens, the reduced Jacobian J used in the projection is perturbed, which can in turn perturb the projected gradient.

To quantify this effect, we measure how often the inferred active set changes under small perturbations. Given an optimization problem with m inequality constraints, we encode the activity pattern as a binary mask $a \in \{0, 1\}^m$. For a reference solution with mask a and a perturbed solution with mask a' , we report the change rate in percent

$$\rho (\%) = \frac{100}{m} \sum_{i=1}^m \mathbb{I}[a_i \neq a'_i]. \quad (47)$$

We generate multiple perturbed costs around $\hat{c}$, recompute the corresponding masks, and average ρ across perturbations. In our LP benchmarks, the average change rates were $\rho = 0.30\%$ for Shortest Path and $\rho = 0.14\%$ for Knapsack. These results validate the stability of active-set identification in our experiments. For problems with many nearly binding constraints, the gradient estimate may be less stable.

Table 11. **Prediction accuracy**. MSE ($\downarrow$), over 5 seeds. Best in **bold** and second best underlined.

(a) LP Benchmarks						(b) Real-world QP	
Task	Method	Deg 2	Deg 4	Deg 6	Deg 8	Method	Normalized MSE
Shortest Path	MSE	0.00$\pm$0.00	0.09$\pm$0.01	0.68$\pm$0.08	4.59$\pm$1.08	MSE	1.105 $\pm$ 0.005
	SPO+	0.31 $\pm$ 0.18	0.61 $\pm$ 0.23	1.55 $\pm$ 0.17	6.24 $\pm$ 1.21	PEAR	<u>1.457 $\pm$ 0.016</u>
	PFYL	1.78 $\pm$ 0.06	1.53 $\pm$ 0.07	1.86 $\pm$ 0.05	5.64 $\pm$ 1.17	QPTH	1.469 $\pm$ 0.024
	DBB	0.10 $\pm$ 0.04	0.20 $\pm$ 0.09	1.14 $\pm$ 0.26	5.65 $\pm$ 1.02	CvxpyLayers	1.493 $\pm$ 0.019
	LAVA	1.16 $\pm$ 0.04	1.19 $\pm$ 0.05	2.37 $\pm$ 0.23	7.89 $\pm$ 1.21		
	PEAR	<u>0.01$\pm$0.01</u>	<u>0.16$\pm$0.06</u>	<u>0.78$\pm$0.09</u>	<u>5.23$\pm$1.18</u>		
Knapsack	MSE	0.18$\pm$0.01	2.67$\pm$0.26	22.58$\pm$3.18	139.42$\pm$21.60		
	SPO+	4.59 $\pm$ 0.09	<u>5.82$\pm$0.35</u>	<u>25.14$\pm$3.10</u>	<u>141.33$\pm$24.10</u>		
	PFYL	8.97 $\pm$ 0.11	8.96 $\pm$ 1.52	36.16 $\pm$ 5.81	181.77 $\pm$ 27.89		
	DBB	12.07 $\pm$ 0.24	11.89 $\pm$ 0.63	29.59 $\pm$ 4.80	144.13 $\pm$ 24.16		
	LAVA	10.97 $\pm$ 0.60	16.27 $\pm$ 1.66	50.65 $\pm$ 5.63	205.00 $\pm$ 31.15		
	PEAR	<u>2.54$\pm$2.53</u>	12.95 $\pm$ 2.01	43.34 $\pm$ 4.96	187.62 $\pm$ 29.71		

Prediction Accuracy and Decision Quality. Existing DFL methods focus on minimizing regret, often at the expense of prediction accuracy measured by MSE. Our framework explains this phenomenon by decomposing prediction error into a decision-relevant tangent component and a decision-irrelevant normal component. Conventional DFL methods implicitly discard the normal component, which can degrade prediction accuracy.

This relationship between regret minimization and prediction accuracy is highly problem-dependent. Since PEAR prioritizes regret reduction, it adaptively allocates learning capacity based on the problem structure. In Shortest Path, where accurate cost prediction of neighboring nodes is inherently linked to identifying optimal paths, PEAR achieves substantially lower

MSE compared to other DFL baselines. In contrast, for Knapsack where relative ranking of item values matters more than precise magnitude estimation, PEAR focuses more on decision quality improvement rather than prediction accuracy.

Our constraint shift experiments further support this observation. In Shortest Path and Portfolio Optimization, where the MSE baseline achieved the best regret under constraint shift, PEAR also exhibited superior prediction accuracy compared to other DFL methods. However, in Knapsack where the MSE baseline suffered significant regret degradation under constraint shifts, PEAR showed higher MSE than other DFL baselines. These results suggest that PEAR implicitly learns to maintain prediction fidelity when it is beneficial for decision quality.

Limitations and Future Directions. While existing DFL methods have achieved strong decision quality, they have operated as black boxes with respect to why regret decreases and how the learning direction is determined. Our work provides a structural characterization that makes these dynamics interpretable through the lens of tangent space projection. Nevertheless, PEAR remains within the DFL paradigm and thus shares its fundamental limitation: an inherent bias toward decision quality at the potential expense of prediction accuracy.

Through our theoretical analysis, we established that the regret gradient lies within the MSE gradient as its projection onto the decision-relevant subspace. The Table 11 results demonstrate that learning only the decision-relevant component can still achieve strong prediction accuracy when problem structure tightly couples prediction and decision quality. Leveraging this decomposition into normal and decision-relevant components to overcome the trade-off between decision quality and prediction accuracy, long viewed as a structural limitation of DFL, remains a promising direction for future work.